Introduction

Programming in Rust · winter 2026/27

Rust is a systems language. This lecture explains why it requires us to say who owns data and who may use it. We will compile a few short programs and read the errors.

How the course works

- Six lecture/lab pairs, plus an optional extension block.
- Lectures and labs are 90 minutes each.
- Each core pair has two ReCodEx homeworks: an executable program and a library.
- The spring advanced Rust course is separate.

Course schedule and materials · *Course material hub*

Work, submission, help

- Use Rust 1.88 for graded work. Check with `rustc --version`.
- Submit `main.rs` for executable tasks and `solution.rs` for library tasks. An assignment may allow additional files.
- Check the course page and each assignment for deadlines and grading.
- Bring questions and attempted solutions to the lab. Use the *course Discord* or contact the instructors.

Install Rust before the next class. Check `cargo --version` and `rustc --version`.

Resource management

A service reads a message and passes it to another thread. Someone must decide how long the message is kept, whether it can change, and when its storage is released.

C and C++ often leave these decisions to the programmer. Garbage collection handles memory reclamation, but it changes the runtime and does not settle all resource or concurrency questions. Rust checks many of these decisions at compile time.

References and mutation

Predict whether this compiles. Then run it. Would reserving extra capacity make a difference?

```
fn main() {
    let mut values = vec![10, 20, 30];
    let first = &values[0];
    values.push(40);
    println!("{first}");
}
```

The compiler’s rule

Result: `push` may move the vector’s elements. The compiler rejects mutation while `first` is still needed, even if this particular `push` would not reallocate. A reference into a vector may become invalid when the vector changes.

This rule rejects some programs that would run safely with today’s inputs. In exchange, safety does not depend on those inputs or on how much memory the vector happens to have reserved.

Discuss ways to keep access to the first value after `push`.

Use an index

An index refers to a position, not an address in the vector's allocation.

```
fn main() {
    let mut values = vec![10, 20, 30];
    let first_index = 0;
    values.push(40);
    println!("{}", values[first_index]);
}
```

Appending leaves earlier indices unchanged. Inserting or removing an element before that position would change what the index refers to.

Ownership

Predict whether both bindings can be used after the assignment. Then run it.

```
fn main() {
    let text = String::from("hello");
    let other = text;
    println!("{text}");
    println!("{other}");
}
```

Moves

Result: `text` cannot be used after the assignment. The `String` moved to `other`; its heap buffer was not copied. Only `other` owns it now.

Rust does not make an implicit copy of a value that owns an allocation.

Copying data

Copying a `String`'s buffer takes time and memory. Copying just its pointer and treating both copies as owners would be invalid.

```
fn main() {
    let text = String::from("hello");
    let other = text.clone();
    println!("{text} / {other}");
}
```

`clone` makes the copy explicit. When the function only needs to read data, borrowing is often simpler.

Borrowed results

Here the function returns a view into its input rather than allocating a new string.

```
fn first_word(input: &str) -> &str {
    input.split_whitespace().next().unwrap_or("")
}

fn main() {
    let line = String::from("systems programming");
    println!("{}", first_word(&line));
}
```

The returned `&str` is valid while the input's storage is valid. No allocation is needed.

Errors as values

`port` returns either a number or a parse error.

```
fn port(input: &str) -> Result<u16, std::num::ParseIntError> {
    input.parse()
}

fn main() {
    for input in ["8080", "not a number"] {
        println!("{input:?}: {:?}", port(input));
    }
}
```

Rust warns when a `Result` is ignored. The caller can handle the error, propagate it, or explicitly discard it. Panics and logic bugs are still possible.

Transferring work to a thread

`move` transfers ownership of `message` into the thread's closure.

```
fn main() {
    let message = String::from("work");
    let handle = std::thread::spawn(move || message.len());
    println!("{}", handle.join().unwrap());
}
```

The thread can use `message` even if the spawning scope ends. `join` waits for its result. We will cover shared state and synchronization later.

Costs and limits

- Safe Rust prevents memory errors of this kind and makes resource ownership explicit.
- Static checking sometimes rejects code that would have run safely. You may need to change the data structure or API.
- Rust does not prevent logic errors or all concurrency errors. `unsafe` code has additional obligations.

Exercise

A function returns a reference into a growable buffer. The caller needs to change the buffer while retaining access to an element.

- Return a copy of the element if that cost is acceptable.
- Keep an index if the planned changes preserve the element's position.
- Finish using the reference before changing the buffer.

Choose one approach and state what the caller may do afterward.

Before next time

Install Rust 1.88, check `rustc --version` and `cargo --version`, and run one example from this lecture. Bring a compiler error that you did not expect.

Next lecture: ownership and borrowing in more detail.